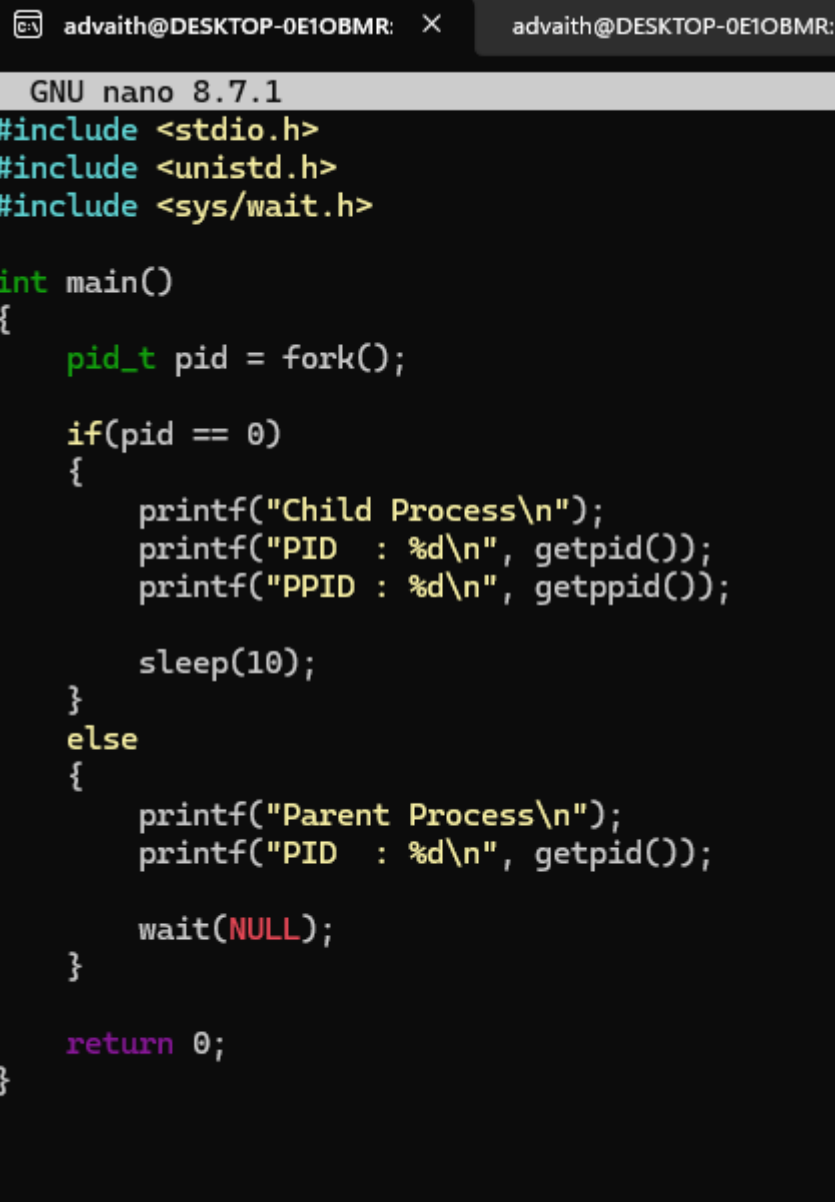


4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

CODE:



The screenshot shows a terminal window with two tabs, both labeled 'advaith@DESKTOP-0E1OBMR:'. The active tab displays the GNU nano 8.7.1 editor. The code in the editor is a C program that demonstrates process synchronization using the wait() function. It includes headers for stdio, unistd, and sys/wait. The main function calls fork() to create a child process. The child process (pid == 0) prints its PID, PPID, and sleeps for 10 seconds. The parent process (else branch) prints its PID and then calls wait(NULL) to wait for the child to complete before returning 0.

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(10);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());

        wait(NULL);
    }

    return 0;
}
```

Output:

```
advaith@DESKTOP-0E10BMR:/mnt/c/Users/HP/Downloads/OSSP-Project$ nano pid.c
advaith@DESKTOP-0E10BMR:/mnt/c/Users/HP/Downloads/OSSP-Project$ gcc pid.c -o
advaith@DESKTOP-0E10BMR:/mnt/c/Users/HP/Downloads/OSSP-Project$ ./program
Parent Process
Child Process
PID : 1091
PID : 1092
PPID : 1091
```